

ADMINISTRATION SERVEUR SECURISE

By : Arnold Edwin FOMEDONG

Objectifs

Ce TP a pour objectifs :

- ➔ Mettre en place l'authentification SSH par clés publiques/privées.
- ➔ Désactiver l'authentification par mot de passe sur le serveur SSH.
- ➔ Présenter et implémenter des méthodes pour protéger la clé privée et réduire le risque si elle est compromise.
- ➔ Proposer une solution plus robuste (certificats SSH, token matériel, 2FA) et expliquer comment l'implémenter.

Partie A - Mise en place de la paire de clés (client)

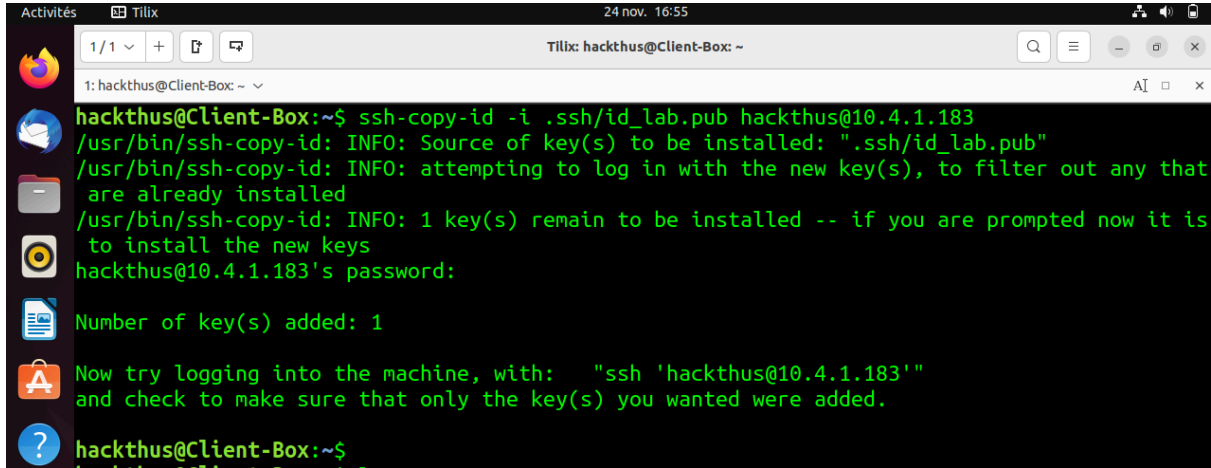
1. Génération d'une paire de clés (sur le client)

Nous générons une paire de clés public et privée sur le client pour nous connecter au serveur

```
hackthus@Client-Box:~$ ssh-keygen -t ed25519 -a 100 -f ~/.ssh/id_lab -C "test@gmail.com"
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/hackthus/.ssh/id_lab
Your public key has been saved in /home/hackthus/.ssh/id_lab.pub
The key fingerprint is:
SHA256:ljjPtYvIY3jsU20EFAVdnqPUPYGiYMXy/jHQboih31M test@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|      O+=.....  |
|    + O 00.O .   |
|  . = +..= O     |
|    . =000 . .   |
|  . +0=S .       |
|  . . 0=E. .     |
|  . + *O+.       |
|    0.O... .     |
|    +++. .       |
+-----[SHA256]-----+
hackthus@Client-Box:~$
```

2. Copier la clé publique sur le serveur

A l'aide ssh-copy-id nous allons copier la clé public nouvellement créé sur le serveur ssh



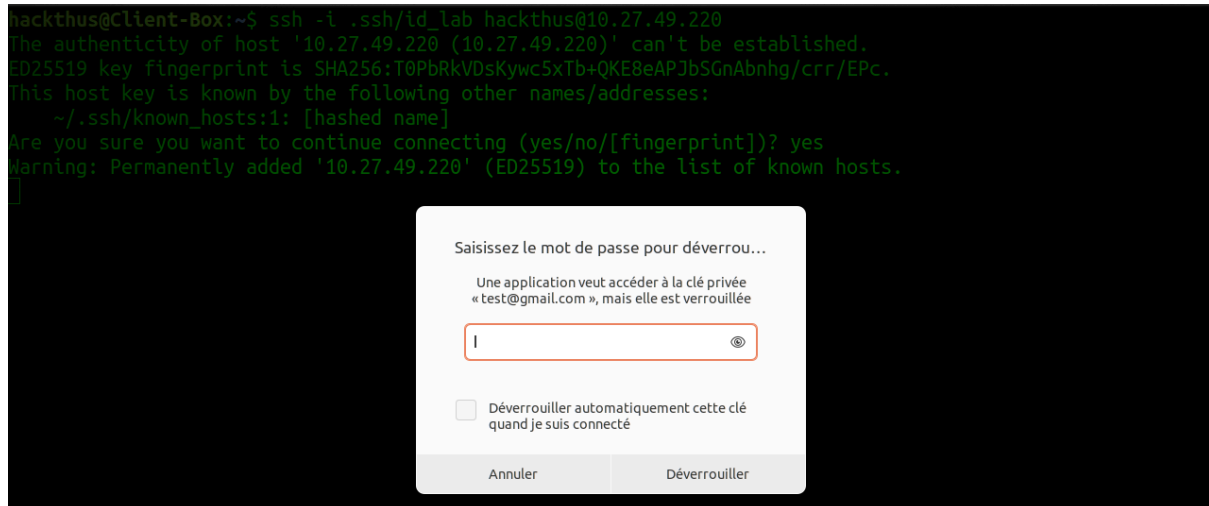
```
hackthus@Client-Box:~$ ssh-copy-id -i .ssh/id_lab.pub hackthus@10.4.1.183
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/id_lab.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is
to install the new keys
hackthus@10.4.1.183's password:
Number of key(s) added: 1
Now try logging into the machine, with: "ssh 'hackthus@10.4.1.183'"
and check to make sure that only the key(s) you wanted were added.
hackthus@Client-Box:~$
```

Ensuite nous vérifions les permissions des fichiers (.ssh et authorized_keys) sur le serveur

```
hackthus@Ubuntu-Box:~$ ls -ld
drwxr-x--- 16 hackthus hackthus 4096 nov. 24 16:45 .
hackthus@Ubuntu-Box:~$ ls -ld .ssh/
drwx----- 2 hackthus hackthus 4096 nov. 24 16:47 .ssh/
hackthus@Ubuntu-Box:~$ ls -al .ssh/authorized_keys
-rw----- 1 hackthus hackthus 96 nov. 24 16:47 .ssh/authorized_keys
hackthus@Ubuntu-Box:~$
```

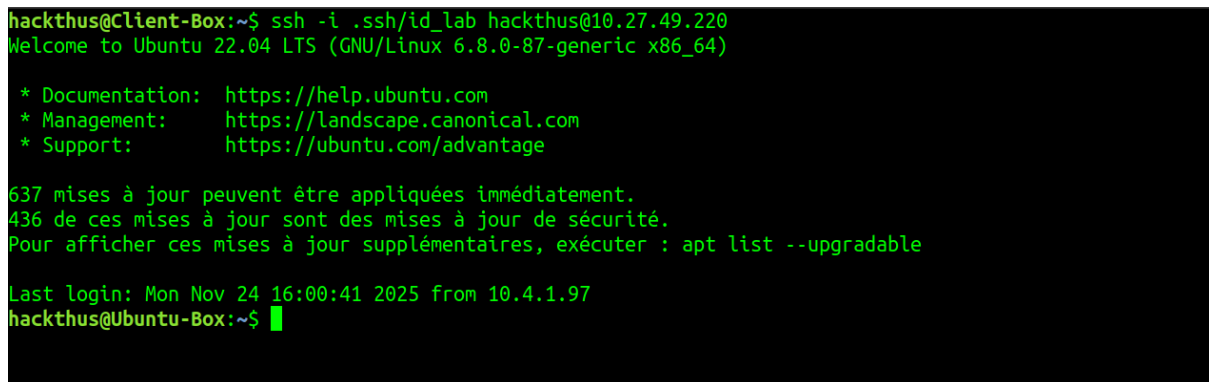
3. Test de connexion

Sur le client nous allons essayer de nous connecter à l'aide de la clé privée que nous avons précédemment créé.



Nous devons fournir notre passphrase pour déverrouiller la clé privéée comme on peut le voir dans l'image ci-dessus.

Une fois la passphrase saisie nous pouvons nous connecter au serveur ssh.



Nous avons pu nous connecter avec succès.

Partie B - Désactivation de l'authentification par mot de passe (serveur)

1. Modifier /etc/ssh/sshd_config

Avant de procéder à la modification de fichier de config sshd_config nous allons d'abord effectuer un backup de ce fichier

```
hackthus@Ubuntu-Box:~$ #sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
hackthus@Ubuntu-Box:~$ ls -al /etc/ssh/
total 560
drwxr-xr-x  4 root root  4096 nov.  26 12:37 .
drwxr-xr-x 129 root root 12288 nov.  26 12:46 ..
-rw-r--r--  1 root root 505426 avril 11  2025 moduli
-rw-r--r--  1 root root  1650 févr. 26  2022 ssh_config
drwxr-xr-x  2 root root  4096 févr. 26  2022 ssh_config.d
-rw-r--r--  1 root root  3236 nov.  26 12:32 sshd_config
-rw-r--r--  1 root root  3254 nov.  26 12:23 sshd_config.bak
drwxr-xr-x  2 root root  4096 avril 11  2025 sshd_config.d
-rw-----  1 root root   505 nov.  24 15:57 ssh_host_ecdsa_key
-rw-r--r--  1 root root   177 nov.  24 15:57 ssh_host_ecdsa_key.pub
-rw-----  1 root root   411 nov.  24 15:57 ssh_host_ed25519_key
-rw-r--r--  1 root root    97 nov.  24 15:57 ssh_host_ed25519_key.pub
-rw-----  1 root root  2602 nov.  24 15:57 ssh_host_rsa_key
-rw-r--r--  1 root root   569 nov.  24 15:57 ssh_host_rsa_key.pub
-rw-r--r--  1 root root   342 déc.   7  2020 ssh_import_id
hackthus@Ubuntu-Box:~$
```

Nous pouvons voir que l'utilisateur toto ne parvient pas à se connecter via password

```
hackthus@Client-Box:~$ ssh toto@10.27.49.220
toto@10.27.49.220: Permission denied (publickey).
hackthus@Client-Box:~$
```

3- Tester si l'authentification par clé fonctionne correctement après modification du fichier de config sshd_config

L'authentification par clé marche correctement comme nous pouvons le voir sur le l'image ci-dessous

```
hackthus@Client-Box:~$ ssh -i .ssh/id_lab hackthus@10.27.49.220
Welcome to Ubuntu 22.04 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

637 mises à jour peuvent être appliquées immédiatement.
436 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Wed Nov 26 12:44:06 2025 from 10.27.49.224
hackthus@Ubuntu-Box:~$
```

Partie C - Durcissement et stratégies pour protéger la clé privée

Utiliser un agent SSH et verrouillage automatique

Charger la clé dans ssh-agent (ou gpg-agent) pour éviter de taper la passphrase trop souvent.

```
hackthus@Client-Box:~$ ssh-add .ssh/id_lab
Enter passphrase for .ssh/id_lab:
Identity added: .ssh/id_lab (test@gmail.com)
hackthus@Client-Box:~$
```

Configurer un timeout d'inactivité pour l'agent (ex : ssh-add-t 3600).

```
hackthus@Client-Box:~$ ssh-add -t 3600 .ssh/id_lab
Enter passphrase for .ssh/id_lab:
Identity added: .ssh/id_lab (test@gmail.com)
Lifetime set to 3600 seconds
hackthus@Client-Box:~$ █
```

4. Utiliser les certificats SSH (meilleure solution pour les environnements d'entreprise)

Plutôt que distribuer des clés publiques statiques, configurer une Autorité de Certification (CA) SSH qui signe des clés utilisateur avec une durée de vie courte. Les avantages :

- Possibilité de révoquer facilement (en expirant les certificats ou en utilisant des listes de révocation).

- Durée de vie courte (ex : 1 heure) : même si la clé privée est volée, le certificat expirera.
- Possibilité d'imposer des contraintes (command=, source-address, etc.)

Créer une clé privée pour l'Autorité de Certification (CA) SSH :

```
hackthus@Client-Box:~$ sudo ssh-keygen -t ed25519 -f /root/ssh_ca -C "ssh CA"
[sudo] Mot de passe de hackthus :
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/ssh_ca
Your public key has been saved in /root/ssh_ca.pub
The key fingerprint is:
SHA256:7dagagnTMbo/eaJ745MPH1BiDcGR6DEybtlnA/2xzMQ ssh CA
The key's randomart image is:
+--[ED25519 256]--+
|  o=+.          |
|o +.o+ E        |
|. +oo+ B o      |
| +.o *o= .      |
|.  +o.oS o      |
|  +... o o      |
|   +o+o o .     |
|   . @+..       |
|  oB+B.         |
+-----[SHA256]-----+
hackthus@Client-Box:~$
```

Cela génère une paire de clés (une privée et une publique) pour l'Autorité de Certification SSH, qui sera utilisée pour signer les clés publiques des utilisateurs.

```
hackthus@Client-Box:~$ sudo ls /root/
snap  ssh_ca  ssh_ca.pub
hackthus@Client-Box:~$ ls .ssh/
id_lab  id_lab.pub  known_hosts  known_hosts.old
hackthus@Client-Box:~$
```

Signature de la clé publique d'un utilisateur :

Utilise la clé privée de la CA pour **signer** la clé publique de l'utilisateur située dans `~/ .ssh/id_lab.pub`) pour une durée de validité de 1 heure.

`ssh-keygen -s /root/ssh_ca -I ammarlab -n hackthus -V +1h /home/hackthus/.ssh/id_lab.pub`

```
root@Client-Box:/home/hackthus# ssh-keygen -s /root/ssh_ca -I ammarlab -n hackthus -V +1h /home/hackthus/.ssh/id_lab.pub
Enter passphrase:
Signed user key /home/hackthus/.ssh/id_lab-cert.pub: id "ammarlab" serial 0 for hackthus valid from 2025-11-26T14:46:00 to 2025-11-26T15:47:25
root@Client-Box:/home/hackthus#
```

La commande s'est terminée avec succès comme présenté dans la figure ci-dessus.

Ce processus génère un **certificat SSH** signé, qui est un fichier contenant la clé publique de l'utilisateur et des informations supplémentaires sur sa validité.

```
hackthus@Client-Box:~$ ls .ssh/
id_lab  id_lab-cert.pub  id_lab.pub  known_hosts  known_hosts.old
hackthus@Client-Box:~$
```

Copier la clé publique de la CA sur les serveurs :

Nous copions la clé publique de la CA sur le serveur à l'aide de scp.

```
root@Client-Box:/home/hackthus# scp /root/ssh_ca.pub hackthus@10.27.49.220
root@Client-Box:/home/hackthus#
```

La commande s'est terminée correctement.

Configurer le serveur pour faire confiance à la CA :

Sur le serveur SSH, nous indiquons à SSH de faire confiance à la clé publique de la CA.

Pour cela, nous ajoutons la ligne suivante (`TrustedUserCAKeys /etc/ssh/ssh_ca.pub`) dans le fichier de configuration `sshd_config` :

```
# PermitTTY no
# ForceCommand cvs server

# Add Trusted Public key CA
TrustedUserCAKeys /etc/ssh/ssh_ca.pub

-- INSERTION --
```

127.1 Bas

Test de la connexion via certificat ssh

Pour tester la connexion SSH, nous spécifions le fichier du **certificat signé** lors de la connexion.

Voici la commande à utiliser pour tester :

```
ssh -i ~/.ssh/id_lab-cert.pub hackthus@10.27.49.220
```



Après avoir renseigné notre passphrase, nous pouvons voir que nous avons réussi à nous connecter au serveur comme le présente l'image ci-dessous

```
hackthus@Client-Box:~$ ssh -i ~/.ssh/id_lab-cert.pub hackthus@10.27.49.220
Welcome to Ubuntu 22.04 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

637 mises à jour peuvent être appliquées immédiatement.
436 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Wed Nov 26 12:51:04 2025 from 10.27.49.224
hackthus@Ubuntu-Box:~$
```

Configurer 2FA (Google Authenticator) pour SSH :

1. Installer Google Authenticator PAM sur le serveur

Pour activer l'authentification par **Google Authenticator** avec SSH, nous allons installer le module PAM correspondant Ubuntu dans notre cas. `sudo apt-get update & sudo apt-get install libpam-google-authenticator`

```
root@Ubuntu-Box:/home/hackthus# sudo apt install libpam-google-authenticator
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
systemd-hwe-hwdb
Veuillez utiliser « sudo apt autoremove » pour le supprimer
```

2. Configurer Google Authenticator pour l'utilisateur

Chaque utilisateur qui souhaite utiliser 2FA devra configurer Google Authenticator sur son propre compte.

Exécutons la commande suivante pour générer une configuration Google Authenticator et une clé secrète OTP : `google-authenticator`

```
hackthus@Ubuntu-Box:~$ google-authenticator
Do you want authentication tokens to be time-based (y/n) y
Warning: pasting the following URL into your browser exposes the OTP secret to Google:
https://www.google.com/chart?chs=280x280&chld=M!0&cht=qr&chl=otpauth://totp/hackthus@Ubuntu-Box:k3Fsecretk308PY7DOH4U25WJ7GLAKI1BY0I26Iissuerk30Uubuntu-Box

Your new secret key is: BFY7DOH4U25WJ7GLAKI1BY0I
Enter code from app (-1 to skip): █
```

Une fois cette étape terminée, nous avons un fichier caché dans le répertoire utilisateur : `~/.google_authenticator`, contenant les informations nécessaires à la génération de l'OTP


```

hackthus@Ubuntu-Box:~$ ls -al
total 96
drwxr-x--- 16 hackthus hackthus 4096 nov. 26 16:04 .
drwxr-xr-x  3 root      root      4096 nov. 24 15:04 ..
-rw-r----- 1 hackthus hackthus 1009 nov. 26 15:09 .bash_history
-rw-r--r--  1 hackthus hackthus  220 nov. 24 15:04 .bash_logout
-rw-r--r--  1 hackthus hackthus 3771 nov. 24 15:04 .bashrc
drwxr-xr-x  2 hackthus hackthus 4096 nov. 24 15:14 Bureau
drwx----- 10 hackthus hackthus 4096 nov. 24 16:00 .cache
drwx----- 11 hackthus hackthus 4096 nov. 24 15:16 .config
drwxr-xr-x  2 hackthus hackthus 4096 nov. 24 15:14 Documents
drwx----- 2 hackthus hackthus 4096 nov. 26 15:15 .gnupg
-r-----  1 hackthus hackthus  136 nov. 26 16:04 .google_authenticator
drwxr-xr-x  2 hackthus hackthus 4096 nov. 24 15:14 Images
-rw-r----- 1 hackthus hackthus   20 nov. 26 12:41 .lessht
drwx----- 3 hackthus hackthus 4096 nov. 24 15:14 .local

```

Laboratoire 3

Objectifs du lab :

- Comprendre la génération de clés SSH publiques/privées.
- Automatiser la rotation des clés via un script et cron job.
- Copier la clé publique automatiquement sur le serveur.
- Seconnecter au serveur en utilisant la clé privée générée.

Partie A -Création du script de génération automatique

Création du script rotate_ssh_key.sh

```

#!/bin/bash

#Variables

USER="testeur"
SERVER="10.27.49.220"
KEY_DIR="$HOME/.ssh/auto_keys"
KEY_FILE="$KEY_DIR/id_ed25519_auto"

mkdir -p $KEY_DIR
ssh-keygen -t ed25519 -f $KEY_FILE -N "" -q

cat "${KEY_FILE}.pub" | ssh $USER@$SERVER 'cat > ~/.ssh/authorized_key_tmp ~/.ssh/authorized_key
& chmod 600 ~/.ssh/authorized_key'

# Nettoyage
rm -f $KEY_DIR/id_ed25519_auto_old
~

```

Ensuite nous lui assignons les droits d'exécution

```
hackthus@Ubuntu-Box:~$ chmod +x rotate_ssh_key.sh
hackthus@Ubuntu-Box:~$
```

Maintenant exécutons le script pour vérifier s'il fonctionne correctement et il est marqué que les fichier existe déjà cela est due aux test effectués précédemment.

```
tatoo@Client-Box:~$ ./rotate_ssh_key.sh
/home/tatoo/.ssh/auto_keys/id_ed25519_auto already exists.
Overwrite (y/n)? y
tatoo@10.27.49.220's password:
tatoo@Client-Box:~$ █
```

Vérification

Enfin, nous vérifions que la nouvelle clé a bien été ajoutée en nous connectant avec l'utilisateur via SSH, mais cette fois en utilisant la clé générée par le script, plutôt qu'un mot de passe

```
tatoo@Client-Box:~$ ssh -i .ssh/auto_keys/id_ed25519_auto tatoo@10.27.49.220
Welcome to Ubuntu 22.04 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

608 mises à jour peuvent être appliquées immédiatement.
427 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Wed Nov 26 21:31:05 2025 from 10.27.49.224
tatoo@Ubuntu-Box:~$
```

Nous avons pu nous connecter via la clé privée générée par le script.

Mais avant ça j'ai rencontré un petit problème le fichier `authorized_keys` sur le serveur était vide, ssh ne parvenais pas à envoyer le clé publique sur le serveur à partir du script car il fallait un mot de passe pour la connexion avant d'effectuer le transfert (ce qui n'est pas défini dans le script), alors a ce niveau j'avais 2 solutions devant moi, soit permettre à l'utilisateur tatoo de se connecter au serveur sans password ce qui est peu recommandable en terme de sécurité. Du coup J'ai donc opté pour la solution 2 transférer la clé publique sur le serveur en utilisant un password via la commande suivante : **cat**

```
~/ssh/auto_keys/id_ed25519_auto.pub | ssh tatoo@10.27.49.220 'cat >
~/ssh/authorized_keys'
```

Partie B — Mise en place du cron job

Ouvrir la crontab pour l'utilisateur client

```
GNU nano 6.2 /tmp/crontab.3IA9jo/crontab *
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/5 * * * * /home/tatoo/.ssh/auto_keys/rotate_ssh_key.sh
```

3. Vérifier que le cron est actif :

```
● cron.service - Regular background program processing daemon
   Loaded: loaded (/lib/systemd/system/cron.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2025-11-26 21:15:12 CET; 1h 7min ago
     Docs: man:cron(8)
  Main PID: 401 (cron)
    Tasks: 1 (limit: 2275)
   Memory: 444.0K
      CPU: 63ms
   CGroup: /system.slice/cron.service
           └─401 /usr/sbin/cron -f -P

nov. 26 21:15:12 Client-Box cron[401]: (CRON) INFO (Running @reboot jobs)
nov. 26 21:17:04 Client-Box CRON[3241]: pam_unix(cron:session): session opened for use>
nov. 26 21:17:04 Client-Box CRON[3242]: (root) CMD ( cd / && run-parts --report /etc>
nov. 26 21:17:04 Client-Box CRON[3241]: pam_unix(cron:session): session closed for use>
nov. 26 21:30:01 Client-Box CRON[11811]: pam_unix(cron:session): session opened for us>
nov. 26 21:30:01 Client-Box CRON[11812]: (root) CMD ([ -x /etc/init.d/anacron ] && if >
nov. 26 21:30:01 Client-Box CRON[11811]: pam_unix(cron:session): session closed for us>
```

En fin nous allons lister la cron planifier pour l'utilisateur hackthus avec la commande `sudo crontab -l -u hackthus`

```
hackthus@Client-Box:~$ sudo crontab -l -u hackthus
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/5 * * * * /home/tatoo/.ssh/auto_keys/rotate_ssh_key.sh
hackthus@Client-Box:~$
```